

Semana 5: La Máquina de Turing — El computador universal más simple

Guía completa de la sesión

Visión general

Duración total	3 horas (2 h cátedra + 1 h laboratorio analógico)
Objetivo central	Que los estudiantes comprendan el modelo de la Máquina de Turing como formalización de “computación”, entiendan la tesis de Church-Turing, y experimenten físicamente el funcionamiento de una MT
Idea ancla	Todo computador — desde un celular hasta un supercomputador — es, en el fondo, equivalente a una máquina absurdamente simple: una cinta, un cabezal, y unas pocas reglas. La potencia no está en la complejidad de la máquina, sino en la sofisticación del programa
Prerrequisito	Semana 4 (algoritmos, propiedades, pseudocódigo)
Materiales	Proyector, pizarra, tiras de papel de 2 metros (6 tiras, una por grupo), marcadores gruesos, tarjetas con tablas de transición impresas, copias de la hoja de laboratorio
Conexión con semanas previas	Semana 1: “la Memoria solo almacena, la ALU solo calcula, la Unidad de Control coordina” — la MT tiene todos estos componentes en su versión más simple. Semana 4: “un algoritmo es un procedimiento finito y definido” — la MT es la <i>formalización</i> de esa idea

PARTE 1: CÁTEDRA (120 minutos)

Bloque A — Apertura: ¿Cuál es la computadora más simple posible? (15 min)

Pregunta al curso:

“En la Semana 1 vimos que un computador tiene CPU, memoria, y entrada/salida. ¿Cuántas de esas partes se pueden eliminar y seguir teniendo algo que ‘computa’?”

Dejar que propongan. Guiar hacia: - ¿Se puede eliminar la memoria? → No, necesitamos almacenar datos. - ¿Se puede eliminar la CPU? → No, necesitamos algo que ejecute instrucciones. - ¿Se puede eliminar la E/S? → Técnicamente sí si la entrada ya está en memoria y la salida queda ahí.

“En 1936, un matemático británico de 24 años se hizo exactamente esta pregunta. Y encontró una respuesta que cambió la historia.”

Bloque B — Alan Turing y el contexto histórico (20 min)

La persona (10 min)

- **Alan Turing** (1912–1954), matemático británico.
- A los 24 años publica “On Computable Numbers” (1936), donde define la Máquina de Turing.
- Durante la WWII, lideró el equipo que descifró Enigma en Bletchley Park — contribución clave a la victoria aliada.
- Pionero de la inteligencia artificial: el “Test de Turing” (1950).

- Perseguido por su homosexualidad bajo las leyes británicas de la época. Murió en 1954. El gobierno británico se disculpó formalmente en 2009.

Nota pedagógica: La historia de Turing tiene un poder narrativo enorme. No es solo un genio abstracto — es una persona con una historia trágica que muestra cómo la sociedad puede destruir a sus propios genios. Esto humaniza la clase y abre una conversación sobre ética e inclusión en la ciencia.

El problema que resolvió (10 min) En 1928, el matemático David Hilbert planteó el **Entscheidungsproblem** (“problema de decisión”):

“¿Existe un procedimiento mecánico que pueda determinar, para cualquier enunciado matemático, si es verdadero o falso?”

Hilbert esperaba que la respuesta fuera sí — un algoritmo universal para toda la matemática.

Turing demostró que la respuesta es **no**. Hay problemas que **ningún** procedimiento mecánico puede resolver. Pero para demostrar eso, primero necesitó definir con precisión qué es un “procedimiento mecánico” — y ahí nació la Máquina de Turing.

Bloque C — El modelo de la Máquina de Turing (35 min)

Los componentes (15 min) Una Máquina de Turing tiene solo **cuatro** partes:

- 1. Una cinta** — infinitamente larga (en teoría), dividida en celdas. Cada celda contiene un símbolo de un alfabeto finito (por ejemplo: 0, 1, y blanco $\square$).
- 2. Un cabezal** — lee el símbolo de la celda actual, puede escribir un nuevo símbolo, y se mueve una posición a la izquierda (L) o a la derecha (R).
- 3. Un conjunto de estados** — la máquina está en un estado a la vez (por ejemplo: q0, q1, q2, ..., qHALT). El estado actual es su “memoria interna”.
- 4. Una tabla de transiciones** — las “instrucciones”. Para cada combinación de (estado actual, símbolo leído), la tabla indica: (símbolo a escribir, dirección de movimiento, nuevo estado).

(estado actual, símbolo leído) → (símbolo a escribir, mover L/R, nuevo estado)

Eso es todo. No hay RAM, no hay disco duro, no hay pantalla. Solo una cinta, un cabezal, estados y reglas.

Comparación con la Semana 1:

Computador Humano (Sem. 1)	Máquina de Turing
Memoria (10 celdas)	Cinta (infinitas celdas)
ALU (calcula)	Cabezal (lee/escribe)
Unidad de Control (lee tarjetas)	Tabla de transiciones
Programador/a (escribe programa)	Quien diseña la tabla
Tarjetas de instrucciones	Filas de la tabla de transiciones

Ejemplo paso a paso (20 min) Problema: sumar 1 a un número binario escrito en la cinta.

Entrada en la cinta: ... $\square$ 1 0 1 1 $\square$... (el número binario 1011 = 11 en decimal)

Resultado esperado: ... $\square$ 1 1 0 0 $\square$... (el número binario 1100 = 12 en decimal)

Tabla de transiciones:

Estado	Lee	Escribe	Mueve	Nuevo estado
q0 (ir al final)	0	0	R	q0
q0	1	1	R	q0
q0	$\square$	$\square$	L	q1
q1 (sumar)	0	1	L	q2
q1	1	0	L	q1

Estado	Lee	Escribe	Mueve	Nuevo estado
q1	□	1	L	q2
q2 (volver al inicio)	0	0	L	q2
q2	1	1	L	q2
q2	□	□	R	qHALT

Ejecución paso a paso en la pizarra:

Estado q0, cinta: □ [1] 0 1 1 □ (cabezal en posición 1)
Lee 1 → escribe 1, mover R → q0

Estado q0, cinta: □ 1 [0] 1 1 □
Lee 0 → escribe 0, mover R → q0

Estado q0, cinta: □ 1 0 [1] 1 □
Lee 1 → escribe 1, mover R → q0

Estado q0, cinta: □ 1 0 1 [1] □
Lee 1 → escribe 1, mover R → q0

Estado q0, cinta: □ 1 0 1 1 [□]
Lee □ → escribe □, mover L → q1

Estado q1, cinta: □ 1 0 1 [1] □
Lee 1 → escribe 0, mover L → q1 (¡carry!)

Estado q1, cinta: □ 1 0 [1] 0 □
Lee 1 → escribe 0, mover L → q1 (¡carry again!)

Estado q1, cinta: □ 1 [0] 0 0 □
Lee 0 → escribe 1, mover L → q2 (carry terminó)

Estado q2, cinta: □ [1] 1 0 0 □
Lee 1 → escribe 1, mover L → q2

Estado q2, cinta: [□] 1 1 0 0 □
Lee □ → escribe □, mover R → qHALT

RESULTADO: □ 1 1 0 0 □ = 1100 en binario = 12 [OK]

Punto clave: la máquina no “sabe” que está sumando. Solo sigue reglas mecánicamente. El “significado” — que esto es una suma — existe solo en la mente de quien diseñó la tabla.

Bloque D — La tesis de Church-Turing (20 min)

La tesis (10 min)

Tesis de Church-Turing: Todo lo que es “computable” (en cualquier sentido razonable de la palabra) puede ser computado por una Máquina de Turing.

No es un teorema demostrado — es una tesis (una afirmación que no se puede probar formalmente, pero que nadie ha logrado refutar en casi 90 años).

Lo que significa: - Tu laptop, tu celular, un supercomputador, y la Máquina de Turing pueden computar **exactamente las mismas cosas**. - La diferencia es solo de **velocidad** — no de capacidad. - No existe una máquina “más poderosa” que una MT. Si la MT no puede resolver un problema, nada puede.

Lo no computable: el problema de la detención (10 min) Turing demostró que hay problemas que **ninguna** MT puede resolver.

El más famoso: el **problema de la detención** (Halting Problem).

“Dado un programa cualquiera y una entrada, ¿se puede determinar si el programa terminará (HALT) o correrá para siempre?”

Turing demostró que **no existe** una MT que resuelva esto para todos los casos.

Ejemplo intuitivo: piensen en un programa que busca el número par más grande. Nunca termina — pero no hay un programa que pueda “mirar” a otro programa y decidir en general si terminará.

Consecuencia: hay límites fundamentales a lo que la computación puede resolver. No todo problema tiene solución algorítmica.

Conexión con conservación: *“¿Se puede crear un algoritmo que prediga con certeza si una especie se extinguirá? Probablemente no — el sistema es demasiado complejo y sensible a condiciones iniciales. Pero se puede crear uno que estime la probabilidad, dado un modelo. La diferencia entre ‘resolver’ y ‘estimar’ es una de las lecciones de la teoría de la computabilidad.”*

Bloque E — ¿Por qué importa todo esto? (15 min)

Tres razones por las que la MT importa hoy **1. Es el fundamento teórico de toda la informática.** Cada lenguaje de programación, cada sistema operativo, cada IA es, en el fondo, equivalente a una MT. La teoría de Turing es a la informática lo que las leyes de Newton son a la ingeniería.

2. Define los límites de lo computable. Saber que hay problemas irresolubles es tan importante como saber resolver problemas. Evita que se pierda tiempo buscando algoritmos que no pueden existir.

3. Es la base para pensar sobre la IA. La Semana 6 hablaremos de LLMs. La pregunta “¿puede una máquina pensar?” fue formulada por Turing en 1950. La MT es el modelo mental que necesitan para tener esa conversación con rigor.

Puente al laboratorio (5 min) *“Ahora van a construir y operar una Máquina de Turing con sus manos. Cada grupo recibirá una tira de papel (la cinta), un marcador (el cabezal), y una tabla de transiciones (el programa). Un estudiante será el cabezal y caminará a lo largo de la tira, leyendo, escribiendo y moviéndose según las reglas. Al final, verificaremos si la máquina hizo su trabajo correctamente.”*

PARTE 2: LABORATORIO ANALÓGICO (60 minutos)

“Sé la Máquina de Turing”

Preparación previa (docente)

Materiales por grupo (×6)

Material	Cantidad	Notas
Tira de papel (cinta)	1 por grupo	Papel de rollo o hojas A4 pegadas con cinta: ~2 metros de largo, dividida en celdas de ~10 cm con líneas verticales (20 celdas). Cada celda debe ser lo suficientemente grande para escribir un símbolo con marcador grueso Para escribir en las celdas Para “borrar” y reescribir celdas (pegar un post-it encima con el nuevo símbolo)
Marcador grueso	1 por grupo	
Corrector líquido o post-its pequeños	1 por grupo	
Tabla de transiciones impresa	1 por grupo (según la tarea asignada)	
Hoja de registro de pasos	1 por grupo	
Flecha de cartón o clip grande	1 por grupo	Para marcar la posición del cabezal en la cinta

Preparación de las cintas Pre-dibujar las celdas en las tiras de papel antes de clase (20 celdas cada una). Dejar todas las celdas en blanco ($\square$ o vacías) — los estudiantes escribirán la entrada al inicio de la actividad.

Tarea A: Sumar 1 a un número binario (grupos 1, 2, 3)

La misma tarea explicada en la cátedra. Los estudiantes ya vieron la ejecución paso a paso, pero ahora deben ejecutarla ellos mismos físicamente.

Entrada en la cinta: $\square \square \square 1 0 1 1 \square \square \square$ (centrada, con blancos a los lados)

Tabla de transiciones: (misma que en clase, entregar impresa)

Estado	Lee	Escribe	Mueve	Nuevo estado
q0	0	0	$\rightarrow$	q0
q0	1	1	$\rightarrow$	q0
q0	$\square$	$\square$	$\leftarrow$	q1
q1	0	1	$\leftarrow$	q2
q1	1	0	$\leftarrow$	q1
q1	$\square$	1	$\leftarrow$	q2
q2	0	0	$\leftarrow$	q2
q2	1	1	$\leftarrow$	q2
q2	$\square$	$\square$	$\rightarrow$	qHALT

Resultado esperado: $\square \square \square 1 1 0 0 \square \square \square$

Variantes por grupo: - Grupo 1: entrada 1011 ($\rightarrow$ 1100) - Grupo 2: entrada 0111 ($\rightarrow$ 1000, con carry que se propaga completamente) - Grupo 3: entrada 1001 ($\rightarrow$ 1010, carry simple)

Tarea B: Reconocer palíndromos binarios (grupos 4, 5, 6)

Problema: determinar si una cadena binaria es un palíndromo (se lee igual al derecho y al revés).

Entrada: una cadena binaria rodeada de blancos.

Estrategia: comparar el primer y último símbolo, marcarlos como “visitados” (X), moverse al siguiente par, y repetir. Si todos los pares coinciden $\rightarrow$ aceptar. Si alguno no $\rightarrow$ rechazar.

Tabla de transiciones:

Estado	Lee	Escribe	Mueve	Nuevo estado	Comentario
q0	0	X	$\rightarrow$	q1	Vio 0 al inicio, buscar 0 al final
q0	1	X	$\rightarrow$	q2	Vio 1 al inicio, buscar 1 al final
q0	X	X	$\rightarrow$	q0	Saltar X ya marcadas
q0	$\square$	$\square$	—	qACCEPT	Todos marcados, es palíndromo
q1	0	0	$\rightarrow$	q1	Avanzar buscando el final
q1	1	1	$\rightarrow$	q1	
q1	X	X	$\rightarrow$	q1	Saltar X
q1	$\square$	$\square$	$\leftarrow$	q3	Llegó al final, retroceder
q3	X	X	$\leftarrow$	q3	Saltar X al final
q3	0	X	$\leftarrow$	q4	¡Coincide! Marcar y volver
q3	1	1	—	qREJECT	No coincide: no es palíndromo
q2	0	0	$\rightarrow$	q2	Avanzar buscando el final
q2	1	1	$\rightarrow$	q2	
q2	X	X	$\rightarrow$	q2	
q2	$\square$	$\square$	$\leftarrow$	q5	Llegó al final
q5	X	X	$\leftarrow$	q5	Saltar X al final
q5	1	X	$\leftarrow$	q4	¡Coincide!
q5	0	0	—	qREJECT	No coincide
q4	0	0	$\leftarrow$	q4	Volver al inicio

Estado	Lee	Escribe	Mueve	Nuevo estado	Comentario
q4	1	1	←	q4	
q4	X	X	→	q0	Volver a empezar

Variantes por grupo: - Grupo 4: entrada 1001 (palíndromo → ACCEPT) - Grupo 5: entrada 1010 (no es palíndromo → REJECT) - Grupo 6: entrada 10101 (palíndromo → ACCEPT)

Desarrollo de la actividad

Roles dentro de cada grupo (5 personas)

Rol	Función
El Cabezal	Se para junto a la cinta, señala la celda actual con la flecha, lee el símbolo, anuncia en voz alta
El Estado	Sostiene una tarjeta con el estado actual (q0, q1...).
El Escritor/a	Cambia la tarjeta cuando la tabla lo indica Escribe en la cinta cuando el cabezal debe escribir (o pega un post-it para “borrar”)
El Buscador/a	Busca en la tabla de transiciones la fila correspondiente y anuncia la instrucción
El Registrador/a	Anota cada paso en la hoja de registro: paso N°, estado, símbolo leído, acción, nuevo estado

Fase 1 — Ejecución guiada (25 min)

1. Cada grupo prepara su cinta (escribe la entrada).
2. El Cabezal se posiciona en la primera celda no vacía.
3. El Estado muestra “q0”.
4. **Ciclo:** El Cabezal lee → El Buscador/a busca en la tabla → anuncia (escribir X, mover derecha, nuevo estado q1) → El Escritor/a escribe → El Cabezal se mueve → El Estado cambia la tarjeta → El Registrador/a anota → repetir.
5. Continuar hasta llegar a qHALT (o qACCEPT / qREJECT).

Regla fundamental: nadie interpreta, nadie anticipa. Se sigue la tabla al pie de la letra, paso por paso. Si alguien dice “ya sé cómo va a terminar”, recordar: *“La Máquina de Turing no piensa. Sigán la tabla.”*

Fase 2 — Verificación y rotación (15 min)

1. Verificar el resultado: ¿la cinta dice lo correcto?
2. Si hubo error: rastrear en la hoja de registro dónde se produjo.
3. Si hay tiempo: los grupos intercambian tareas (los de suma hacen palíndromos y viceversa).

Fase 3 — Discusión plenaria (20 min) Preguntas guía:

1. “¿Cuántos pasos necesitaron para sumar 1 a un número de 4 dígitos?” → Contar en la hoja de registro. Relacionar con la complejidad: para n dígitos, ¿cuántos pasos se necesitan?
2. “¿El Cabezal necesitó ‘entender’ que estaba sumando?” → No. Solo leía y seguía reglas. La máquina no sabe qué hace. El significado está en el diseño de la tabla. → **Conexión con Semana 1:** igual que la Memoria y la ALU en el Computador Humano.
3. “¿Podrían escribir una tabla de transiciones para cualquier algoritmo del que hablamos en la Semana 4?” → En principio, sí. Eso es lo que dice la tesis de Church-Turing: todo algoritmo se puede implementar como una MT. Sería muy tedioso, pero es posible.
4. “Si esta máquina tan simple puede computar todo lo computable, ¿qué agrega un computador moderno?” → **Velocidad y comodidad.** Un computador no puede hacer nada que una MT no pueda — pero lo hace billones de veces más rápido. Es como la diferencia entre caminar y volar: el destino es el mismo, pero el tiempo cambia todo.

5. “¿Qué tiene que ver con la inteligencia artificial?” → Anticipo de Semana 6: un LLM es, en el fondo, una MT (o un programa ejecutándose en una). Cuando ChatGPT “escribe un texto”, está siguiendo reglas (muy complejas) sobre una entrada (tu prompt). La pregunta de Turing: ¿es eso “pensar”?

Notas pedagógicas

Adaptación para auditorio con bancas fijas

La cinta se coloca en el **suelo del pasillo frente a las primeras bancas**, o a lo largo de una **fila de bancas vacía**. El Cabezal camina al lado de la cinta. Los demás integrantes se sientan en las bancas adyacentes con sus materiales.

Si el espacio es muy limitado: la cinta se coloca **sobre la banca** (horizontal) y el Cabezal usa el dedo o la flecha de cartón para señalar la celda actual, sin necesidad de caminar.

El error como herramienta

La Tarea B (palíndromos) es deliberadamente más compleja que la Tarea A. Es probable que los grupos 4–6 cometan errores. Eso es valioso: - Un error en el paso 8 de 20 es difícil de rastrear → introduce la idea de **debugging**. - Pregunta: “¿Cómo sabrían dónde se equivocaron sin la hoja de registro?” → La hoja de registro es un **log**, un concepto que aparece en programación (debugging) y en ecología (bitácora de campo).

Lo que NO se pretende en esta clase

- **No se pretende que memoricen tablas de transiciones.** El objetivo es que entiendan el *modelo*, no que puedan diseñar MTs complejas.
- **No se entra en la demostración formal del Halting Problem.** Se presenta como resultado conceptual.
- **No se distingue entre MT deterministas y no deterministas.** Eso es material de teoría de computación avanzada.

Conexión con el resto del curso

Concepto de esta semana	Dónde se profundiza
La MT como modelo de computación	Semana 6 (¿los LLMs “piensan” o siguen reglas?)
Tabla de transiciones como programa	Semana 8 (un programa en Python es un conjunto de reglas)
Lo no computable (Halting Problem)	Semana 6 (límites de la IA)
El cabezal lee/escribe en la cinta	Semana 8 (variables como celdas de memoria)
Estados como “memoria interna”	Semana 10 (condicionales como cambio de estado)
Debugging con la hoja de registro	Semana 8–16 (debugging en Python)